



Zero Touch Node

Setup Guide for Dynamo Revit

Step-by-Step Development Environment Configuration

C# · Visual Studio · Revit API · Dynamo

© 2025 BIMEra. All Rights Reserved.

Introduction

A Zero Touch Node is a custom Dynamo node written in C#. Unlike Python nodes, Zero Touch Nodes are compiled into a DLL and loaded by Dynamo at startup, giving you full access to the Revit API, better performance, and a professional development workflow.

This guide walks you through the complete setup process from installing Visual Studio to seeing your custom node appear inside Dynamo Revit.

Prerequisites	Required Version
Autodesk Revit	2020 or later (2024/2025 recommended)
Dynamo	Installed with Revit (Dynamo Revit 2.x / 3.x)
Visual Studio	Community 202x (free)
.NET Framework	4.8 (for Revit 2024 and below)
.NET	8.0 (for Revit 2025 and above)

Step 1 — Download & Install Visual Studio Community

Visual Studio Community is Microsoft's free, fully-featured IDE. It is the recommended environment for Zero Touch Node development.

1.1 Download Visual Studio

1. Open your browser and navigate to: <https://visualstudio.microsoft.com/downloads/>
2. Under the "Community" edition (free), click the Download button.
3. Run the installer once the download completes.

1.2 Select the Required Workload

The installer will ask which workloads to install. You must select the .NET desktop development workload — this includes the compilers and libraries needed for building class libraries.

- 1 In the Visual Studio Installer, click "Modify" (or select workloads on first install).
- 2 Check the box for ".NET desktop development".

- 3 In the right-hand panel, verify that ".NET Framework 4.8 development tools" is included.
- 4 Click "Install" and wait for the process to complete.

✓ TIP

If Visual Studio was already installed, open the Visual Studio Installer from the Start Menu, click Modify on your installation, and add the .NET desktop development workload.

Step 2 — Create a C# Class Library Project

The project type depends on which version of Revit you are targeting. Read the table below carefully before proceeding.

Revit Version	Project Template	.NET Version
Revit 2024 and below	Class Library (.NET Framework)	.NET Framework 4.8
Revit 2025 and above	Class Library (.NET)	.NET 8.0

2.1 Create the Project

1. Open Visual Studio and select "Create a new project".
2. In the search box, type "Class Library" to filter templates.
3. Select the correct template for your Revit version (see table above).
4. Name your project (e.g., ZeroTouchNodeDemo) and choose a save location.
5. Click "Create".

⚠ NOTE

Do NOT use the same name for two different packages. Dynamo identifies packages by folder name, which is derived from your project name. Duplicate names will cause one package to overwrite the other.

2.2 Delete the Default Class File

Visual Studio auto-generates a Class1.cs file. You can rename it to your own class file with a meaningful name (e.g., RevitInteraction.cs).

Step 3 — Add the Post-Build Script

The post-build script automatically copies your compiled DLL and supporting files into the correct Dynamo packages folder every time you build the project. Without this, you would have to copy files manually after every build.

3.1 Unload the Project

6. In the Solution Explorer, right-click your project name.
7. Select "Unload Project". The project will appear greyed out with a "(unloaded)" label.
8. Right-click the unloaded project again.
9. Select "Edit [ProjectName].csproj". This opens the raw XML file.

3.2 Paste the Post-Build Target

Scroll to the bottom of the .csproj file. Just before the closing `</Project>` tag, paste the following XML block:

```
<Target Name="PostBuild" AfterTargets="Build">
  <!-- Copy the package to the Dynamo package root directory -->
  <GetAssemblyIdentity AssemblyFiles="$(OutDir)\$(TargetName).dll">
    <Output TaskParameter="Assemblies" ItemName="AssemblyInfo" />
  </GetAssemblyIdentity>
  <ItemGroup>
    <SourceDlls Include="$(TargetDir)*.dll" />
    <SourcePdb Include="$(TargetDir)*.pdb" />
    <SourceXmls Include="$(TargetDir)*.xml" />
    <SourcePkg Include="pkg.json" />
  </ItemGroup>
  <PropertyGroup>
    <DynamoPackageRoot>
      $(AppData)\Dynamo\Dynamo Revit\3.3\packages\$(MSBuildProjectName)
    </DynamoPackageRoot>
    <DynamoBin>$(DynamoPackageRoot)\bin</DynamoBin>
  </PropertyGroup>
  <!-- Remove existing bin and recreate to ensure clean copy -->
  <RemoveDir Directories="$(DynamoBin)" Condition="Exists('$(DynamoBin)') " />
  <MakeDir Directories="$(DynamoBin)" />
  <!-- Copy files, skip unchanged to speed up incremental builds -->
  <Copy SourceFiles="@ (SourceDlls)" DestinationFolder="$(DynamoBin)"
    SkipUnchangedFiles="true" />
  <Copy SourceFiles="@ (SourcePdb)" DestinationFolder="$(DynamoBin)"
    SkipUnchangedFiles="true" />
  <Copy SourceFiles="@ (SourceXmls)" DestinationFolder="$(DynamoBin)"
    SkipUnchangedFiles="true" />
  <Copy SourceFiles="@ (SourcePkg)" DestinationFolder="$(DynamoPackageRoot)"
    SkipUnchangedFiles="true" />
</Target>
```

NOTE

The path in `DynamoPackageRoot` contains 3.3 — this is the Dynamo version, not the Revit version. If you are using a different Dynamo version, update this path accordingly (e.g., 3.2 or 3.4).

3.3 Reload the Project

10. Save the `.csproj` file.
11. In the Solution Explorer, right-click the project.
12. Select "Reload Project".

3.4 Add the `pkg.json` File

The post-build script copies a `pkg.json` file to the Dynamo package root. You must create this file manually in the same folder as your `.csproj` file.

Right-click the project in Solution Explorer, select Add > New Item, and create a file named `pkg.json`. Paste the following content:

```
{
  "license": "",
  "file_hash": null,
  "name": "YourPackageName",
  "version": "1.0.0",
  "description": "Your package description",
  "group": "",
  "keywords": null,
  "dependencies": [],
  "contents": "",
  "engine_version": "3.3.0",
  "engine": "dynamo",
  "engine_metadata": "",
  "site_url": "",
  "repository_url": "",
  "contains_binaries": true,
  "node_libraries": [
    "YourProjectName, Version=1.0.0.0, Culture=neutral"
  ]
}
```

TIP

Set "name" in `pkg.json` to match your project name exactly. This is what Dynamo displays as the package name in the Library panel.

Step 4 — Add DLL References

Your project needs to reference several DLLs from Revit and Dynamo to access their APIs. These references tell the compiler which classes and methods are available at build time.

The paths below are for Revit 2025. If you are targeting a different version, replace 2025 with your version number in each path.

4.1 Required DLL Paths

RevitServices.dll	Dynamo-Revit bridge services
RevitNodes.dll	Dynamo Revit node wrappers
ProtoGeometry.dll	Dynamo geometry types
RevitAPI.dll	Core Revit API
RevitAPIUI.dll	Revit UI API



```

C:\Program Files\Autodesk\Revit
2025\AddIns\DynamoForRevit\Revit\RevitServices.dll
C:\Program Files\Autodesk\Revit 2025\AddIns\DynamoForRevit\Revit\RevitNodes.dll
C:\Program Files\Autodesk\Revit 2025\AddIns\DynamoForRevit\ProtoGeometry.dll
C:\Program Files\Autodesk\Revit 2025\RevitAPI.dll
C:\Program Files\Autodesk\Revit 2025\RevitAPIUI.dll

```

4.2 How to Add a Reference

13. In the Solution Explorer, right-click "References" (for .NET Framework) or "Dependencies" (for .NET).
14. "Add Reference..." or "Add Project Reference..." > Browse.
15. Navigate to the path shown above and select the DLL.
16. Click OK to add it.
17. Repeat for all five DLLs.

4.3 Set Copy Local = False

This is a critical step. By default, Visual Studio copies referenced DLLs into your output folder. Since Revit and Dynamo already have these DLLs loaded at runtime, copying them would bloat your package and can cause version conflicts.

18. In the Solution Explorer, expand the References or Dependencies node.
19. Click on one of the five DLLs you just added (e.g., RevitAPI).

20. In the Properties panel at the bottom of Solution Explorer, find "Copy Local".
21. Change the value from True to "No" / "False".
22. Repeat for all five references.

The screenshot below shows the Solution Explorer with all five references added and the Properties panel confirming Copy Local is set to No:

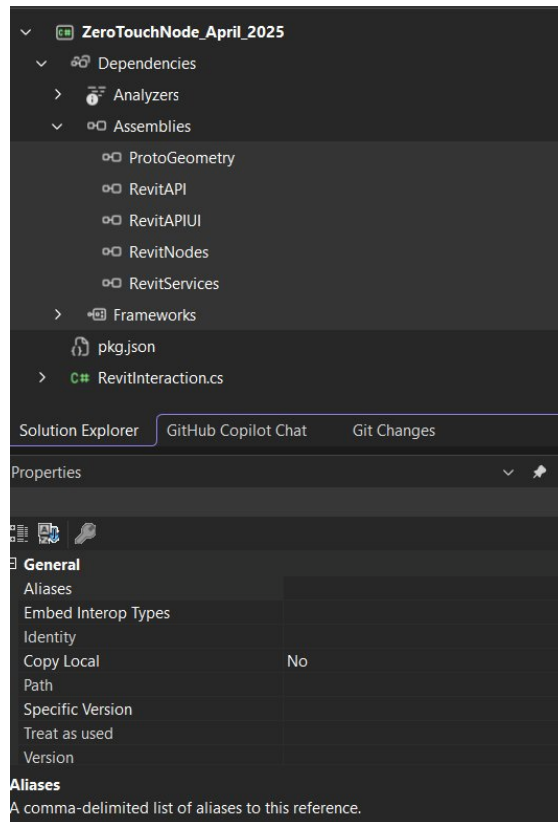


Figure 1: Solution Explorer showing DLL references with Copy Local = No

NOTE

If you forget to set Copy Local to No, Dynamo may load the wrong version of the DLL and throw errors or behave unexpectedly. Always double-check all five references.

Step 5 — Write Your First Zero Touch Node

A Zero Touch Node is simply a public static method inside a public class. Dynamo discovers and exposes it automatically. Below is a minimal example that retrieves all walls from the active Revit document.

```
using System.Collections.Generic;
using Autodesk.Revit.DB;
using RevitServices.Persistence;
```

```
namespace ZeroTouchNode_April_2025
{
    public class RevitInteraction
    {
        /// <summary>
        /// Gets all walls in the active Revit document.
        /// </summary>
        /// <returns>List of Wall elements</returns>
        public static List<Wall> GetAllWalls()
        {
            Document doc = DocumentManager.Instance.CurrentDBDocument;
            FilteredElementCollector collector =
                new FilteredElementCollector(doc)
                    .OfClass(typeof(Wall));
            return collector.Cast<Wall>().ToList();
        }
    }
}
```

✓ TIP

Every public static method in your public class becomes a node. The method name becomes the node name. XML documentation comments (///) become the node tooltip in Dynamo.

Step 6 — Build the Project

Once your code is ready and all references are set, build the project. The post-build script will handle copying everything to the right place.

6.1 Build Steps

23. From the menu bar, select Build > Build Solution (or press Ctrl + Shift + B).
24. Watch the Output panel for any errors or warnings.
25. If the build succeeds, the post-build script runs automatically.

6.2 What the Post-Build Script Does

- Removes the existing bin folder from the Dynamo package directory (ensures a clean copy).
- Creates a fresh bin folder inside your package folder.
- Copies all .dll, .pdb, and .xml files from the build output to the bin folder.
- Copies pkg.json to the package root.

6.3 Package Location After Build

The compiled package will be placed at this path:


```
%AppData%\Dynamo\Dynamo Revit\3.3\packages\[YourProjectName]\
├── pkg.json
├── bin\
│   ├── [YourProjectName].dll
│   ├── [YourProjectName].pdb
│   └── [YourProjectName].xml
```

NOTE

If the build fails with an error about copying files, check that Dynamo Revit is not currently running. Dynamo locks the DLL files while it is open, preventing the copy step from completing.

Step 7 — See Your Node in Dynamo Revit

After a successful build, your custom node is ready to use inside Dynamo Revit. Follow these steps to locate it.

26. Open Revit and load or create a project.
27. Launch Dynamo from the Manage tab > Visual Programming > Dynamo.
28. In the Dynamo window, open the Library panel on the left side.
29. Look for a group matching your project name. Expand it to see your nodes.
30. Alternatively, type your class or method name in the Library search bar.
31. Drag the node onto the canvas and wire it up to test it.

TIP

If the node does not appear, close Dynamo, rebuild the project, and reopen Dynamo. Check the Dynamo package folder to confirm the DLL was copied correctly.

Troubleshooting

Problem	Solution
Node not appearing in Dynamo	Verify the DLL was copied to the packages folder. Confirm pkg.json has the correct project name in node_libraries. Restart Dynamo.
Build error: file in use	Close Dynamo before building. Dynamo locks DLL files while running.
MissingMethodException at runtime	Set Copy Local = No on all Revit/Dynamo references to avoid DLL version conflicts.

Wrong .NET framework version	Check your .csproj target framework. Use net48 for Revit 2024 and below, net8.0-windows for Revit 2025 and above.
pkg.json not found error	Ensure pkg.json is in the same directory as the .csproj and is included in the project (Build Action: Content or None).

Quick Reference Summary

#	Step	Key Action
1	Install Visual Studio	Download Community edition, select .NET desktop development workload
2	Create Class Library	Use .NET Framework for Revit ≤ 2024 , use .NET for Revit ≥ 2025
3	Post-Build Script	Unload project, edit .csproj, add Target block, reload project
4	Add References	Add 5 DLLs, set Copy Local = No on all
5	Write Your Node	Public static method in a public class
6	Build	Ctrl+Shift+B — script auto-copies to Dynamo packages folder
7	Test in Dynamo	Open Dynamo, find node in Library or search by name

BIMEra | Empowering BIM Professionals
Built for Dynamo students by BIMEra. Happy coding.